



Lesson 4: Classification and Evaluation Metrics

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

Before we start

- Exercise 3, which we discuss now
- Ridge, Lasso, and the choice of the penalty strength

Today: from a number to a decision

- Logistic regression: the model and why the cost had to change
- The confusion matrix, and why accuracy lies
- Precision, recall, F1, and the threshold nobody chose
- Receiver operating characteristic (ROC) curves, and where they mislead
- Choosing a threshold from money

The question has changed

- Lesson 3: given a house, **how many euros?**
- Lesson 4: given a disk drive, **will it fail?**

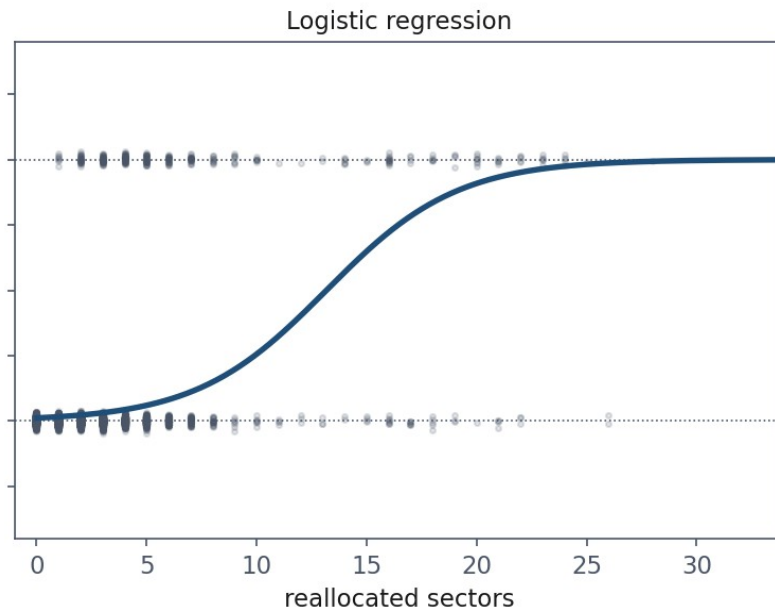
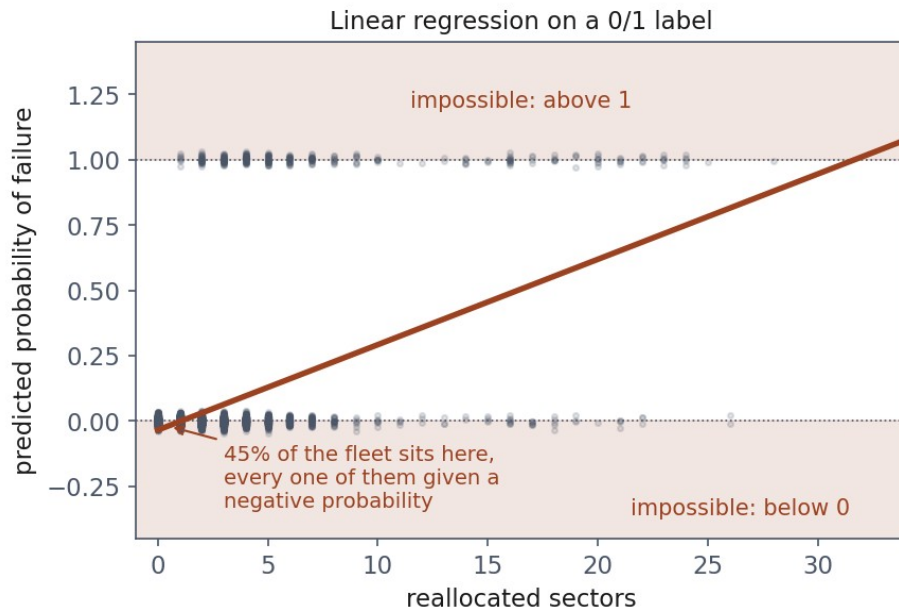
The fleet

- **8,000 disk drives** in a data centre
- Six SMART counters each: Self-Monitoring, Analysis and Reporting Technology
- Label: did it fail within thirty days?
- `train_test_split(..., stratify=y)`: 6,000 to train, 2,000 to test

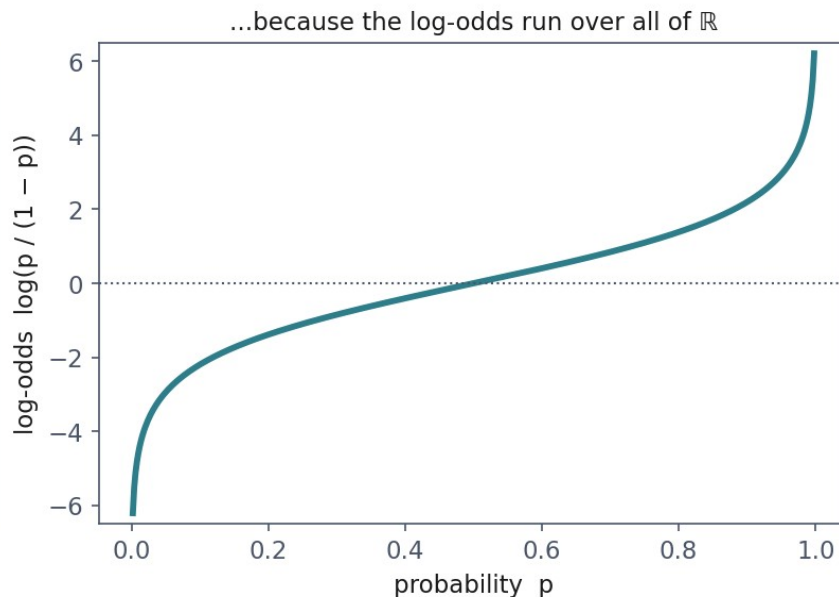
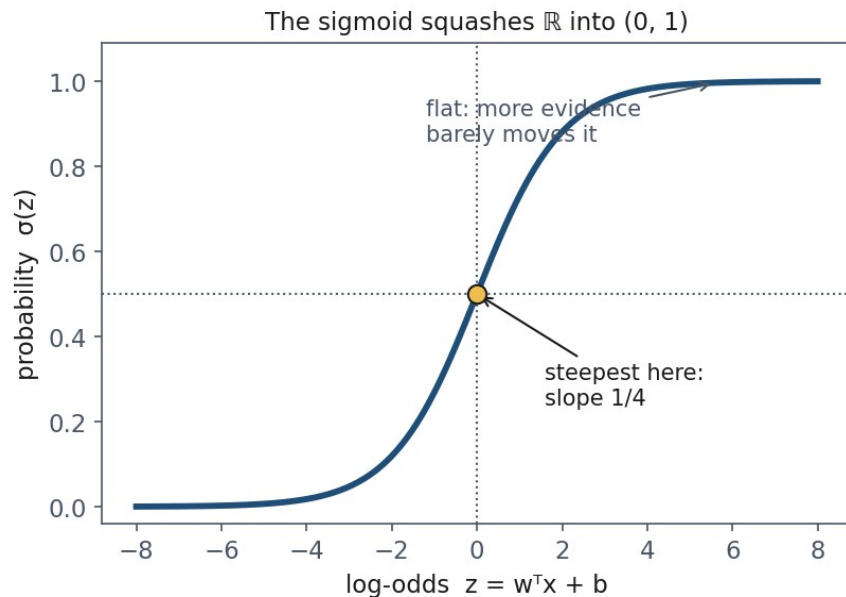
306

- **306 of the 8,000 drives failed.** 3.8% of the fleet
- Which means 96.2% did not

What goes wrong if we just fit a line



The sigmoid



The model, in one line

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

What the coefficients mean

- The model is linear in the log-odds, $\log(p \div (1 - p))$, so `exp(coefficient)` multiplies the **odds**, $p \div (1 - p)$
- Features standardised, so one unit = one standard deviation
- A coefficient of 1.80 means **six times the odds** of failing
- In code: `make_pipeline(StandardScaler(), LogisticRegression())`

The truth, and what we recovered

Feature	True	Fitted	Odds ×
reallocated_sectors	1.80	1.66	5.28
spin_retry_count	1.15	1.17	3.22
read_error_rate	0.85	0.87	2.39
temperature_c	0.45	0.47	1.59
power_on_hours	0.35	0.38	1.46
seek_error_rate	0.00	0.02	1.03

The intercept is the average drive's chance

- Fitted intercept: **-6.09**
- $\sigma(-6.09) = 0.0023$: a 0.2% chance for a drive average on every counter
- The fleet fails at 3.8%: the failures come from the few drives far above average

Fitting means minimising something, but what?

- Squared error worked for lesson 3
- Why not just apply it to the sigmoid's output?

Maximum likelihood, in one sentence

- The parameters w and b are a knob: each setting gives a probability to what actually happened
- **Turn the knob so that what actually happened is least surprising**

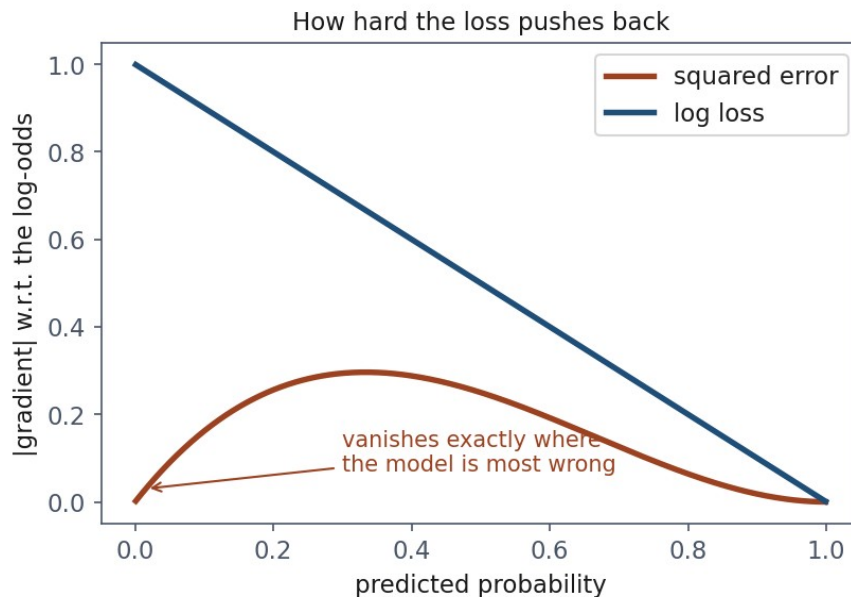
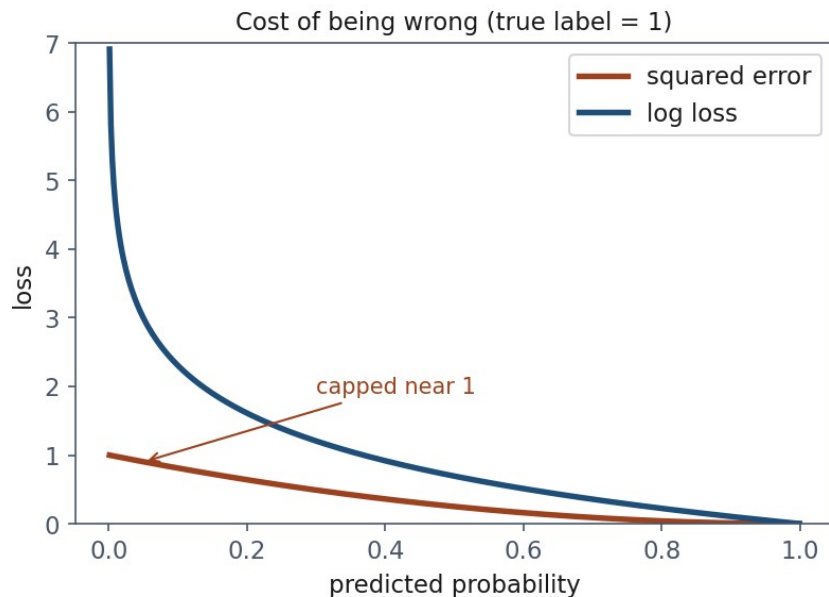
Cross-entropy, also called log loss

$$L = - [y \log p + (1 - y) \log(1 - p)]$$

The gradient is the one you already know

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

So why not squared error?



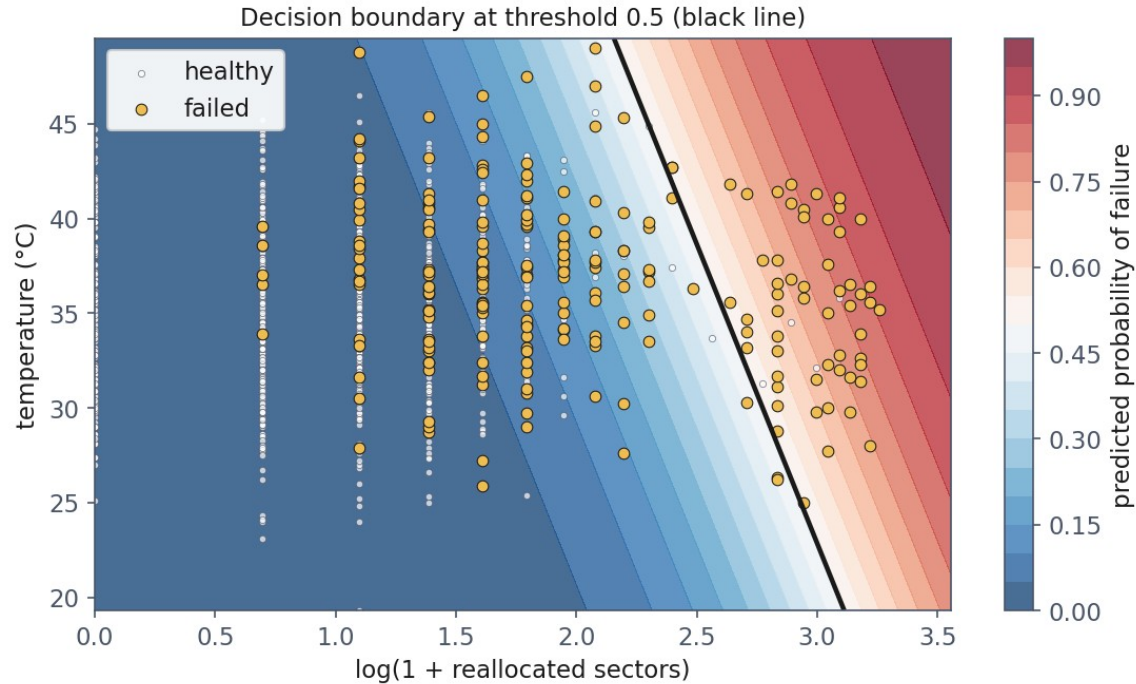
And it is convex, which squared error is not

- Log loss is **convex** in the weights, so any local minimum is global
- Squared error through a sigmoid is not

Notebook 1, live

section, and its slides	you call (→ the number to come back with)
a line on a 0/1 label, 7	<code>LinearRegression</code> → 3,606 negative probabilities
odds, and why not squared error, 8-18	NumPy only: the sigmoid, the two losses, their gradients
gradient descent, 16	<code>train_test_split</code> , <code>StandardScaler</code> , fifteen lines of NumPy → log loss 0.6931 to 0.0692
the decision boundary, 20	<code>make_pipeline(StandardScaler(), LogisticRegression())</code> on two features
the truth, recovered, 11-12	<code>LogisticRegression</code> , <code>.coef_</code> , <code>.intercept_</code> → <code>seek_error_rate</code> 0.02, intercept -6.09

The decision boundary



Break

- Ten minutes

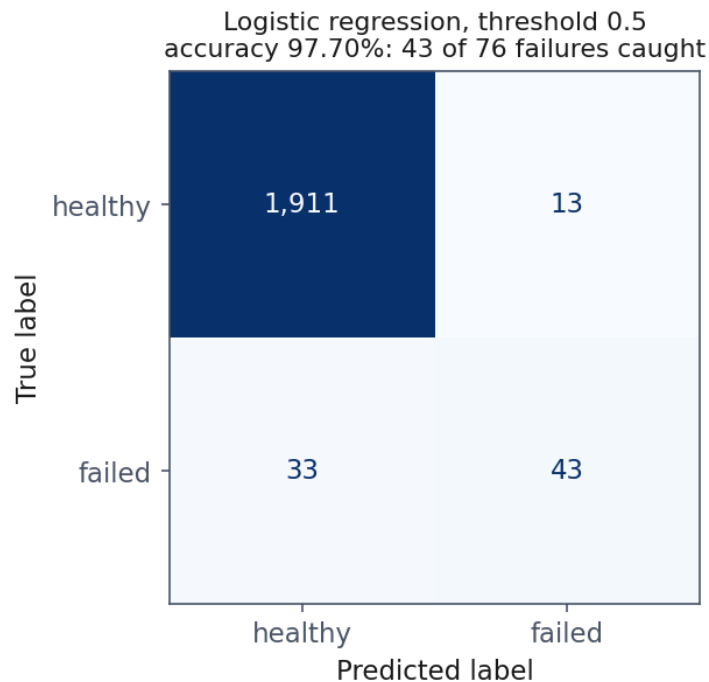
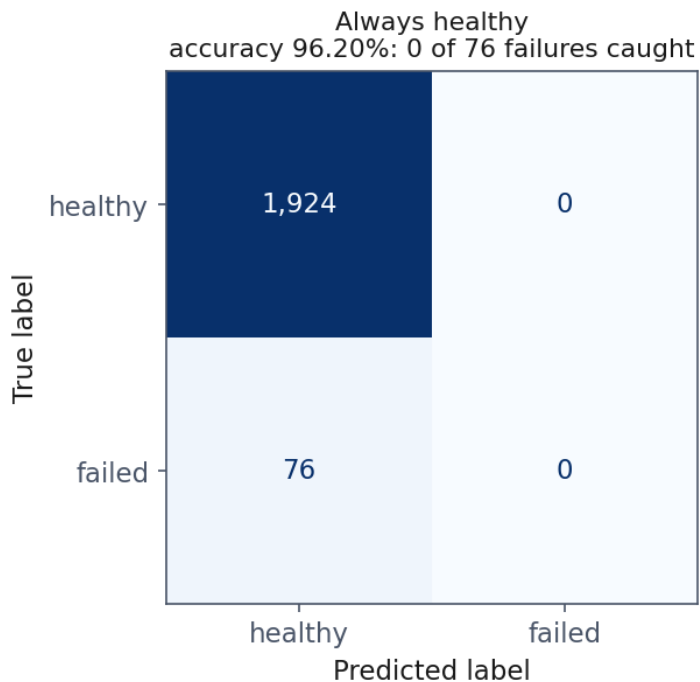
Two models

Model	Accuracy	Failures caught
Predict “healthy” for everything	96.20%	0 of 76
Logistic regression, threshold 0.5	97.70%	43 of 76

Why everyone makes this mistake once

- Accuracy is the fraction of predictions that were right
- On a **balanced** problem that is exactly what you want
- The interesting class is almost always the **rare** one

Four numbers instead of one



The four outcomes, and what they cost

- **False positive:** we replace a healthy drive. A technician's hour, €140
- **False negative:** a drive dies in service. Callout, rebuild, risk, €2,600

Precision and recall: look at the denominator

Precision = $TP / (TP + FP)$
“of the alarms we raised”

	predicted healthy	predicted failing
actually healthy	TN 1911	FP 13
actually failed	FN 33	TP 43

Recall = $TP / (TP + FN)$
“of the drives that failed”

	predicted healthy	predicted failing
actually healthy	TN 1911	FP 13
actually failed	FN 33	TP 43

Our model, in those terms

- True and false positives and negatives: TP, FP, FN, TN
- Precision = $TP \div (TP + FP) = 43 \div 56 = \mathbf{0.768}$: of the drives flagged
- Recall = $TP \div (TP + FN) = 43 \div 76 = \mathbf{0.566}$: of the drives that failed
- Specificity = $TN \div (TN + FP) = 1,911 \div 1,924 = 0.993$
- In code: `precision_score, recall_score`

F1: why the harmonic mean

Model	Precision	Recall	Arithmetic mean	Harmonic, $F1 = 2PR \div (P + R)$
Flag every drive	0.038	1.000	0.519	0.073

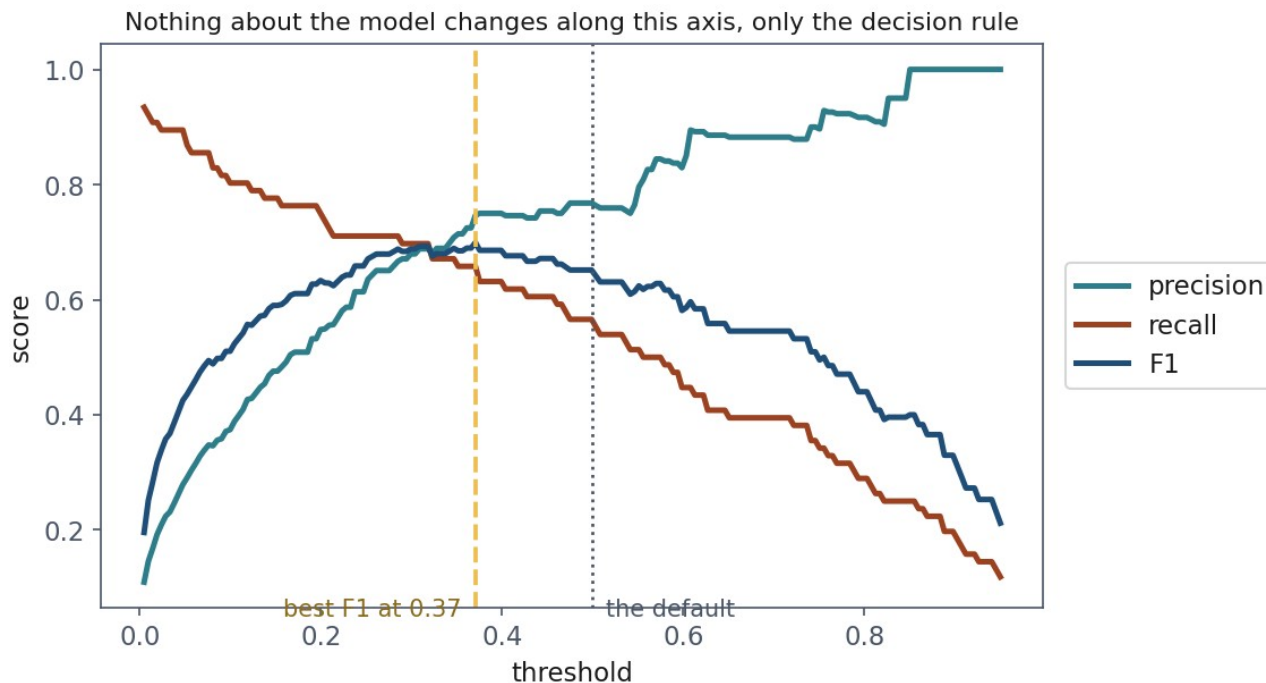
Read the minority class, not the average

- weighted avg F1: **0.975** (dominated by the healthy class)
- macro avg F1: **0.820** (both classes get an equal say)
- In code: `classification_report(y_test, y_pred)` prints both rows

Notebook 2, live

section, and its slides	you call (→ the number to come back with)
the number that should worry you, 22	<code>accuracy_score</code> → 96.20% for “always healthy”
the confusion matrix, 24	counts by hand, then <code>confusion_matrix</code> , <code>ConfusionMatrixDisplay</code> → TP 43
precision and recall, 26-29	<code>precision_score</code> , <code>recall_score</code> , <code>f1_score</code> , <code>classification_report</code>
the threshold sweep, 31-32	<code>predict_proba</code> , then the metrics at each threshold
the precision-recall curve, 33	<code>precision_recall_curve</code> , <code>average_precision_score</code> → 0.717 against 0.038

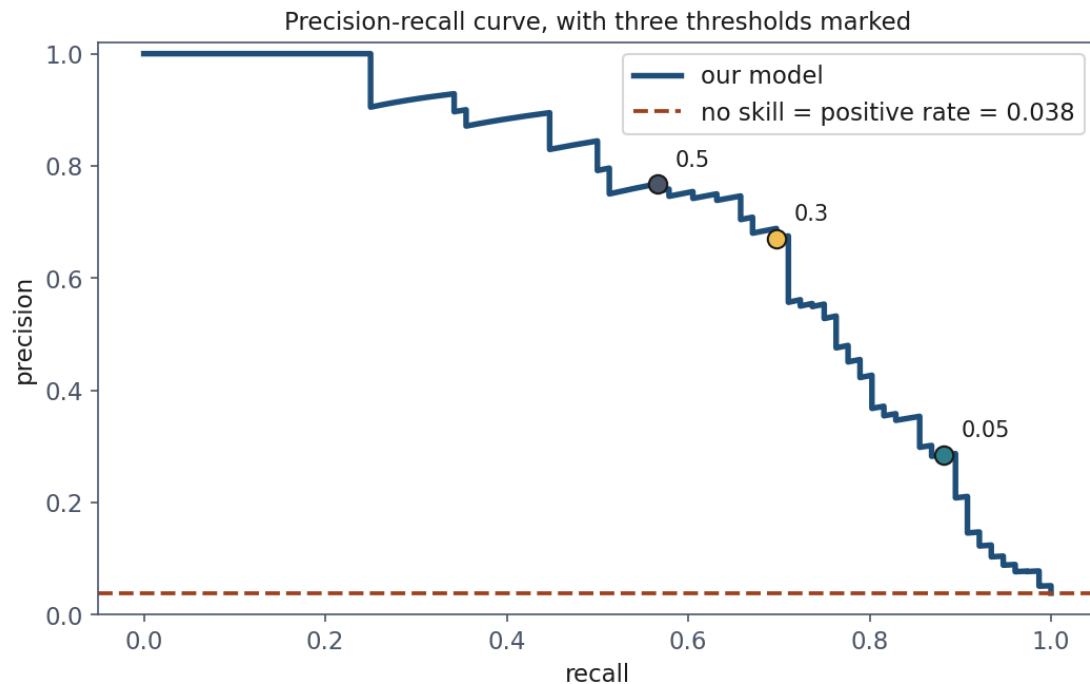
The threshold is a choice nobody made



What the sweep actually costs

Threshold	TP	FP	FN	Precision	Recall	Accuracy
0.02	69	284	7	0.195	0.908	0.855
0.10	61	102	15	0.374	0.803	0.942
0.30	53	25	23	0.679	0.697	0.976
0.50	43	13	33	0.768	0.566	0.977
0.90	14	0	62	1.000	0.184	0.969

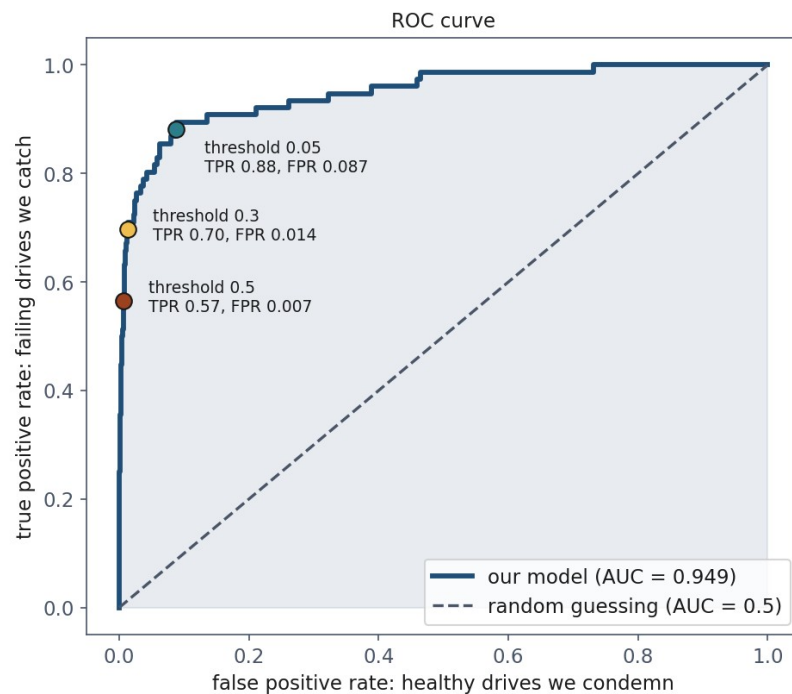
The precision-recall curve



Comparing models without choosing a threshold

- We keep evaluating at one threshold, but which?
- What if we score the whole **ranking** instead?

The ROC curve



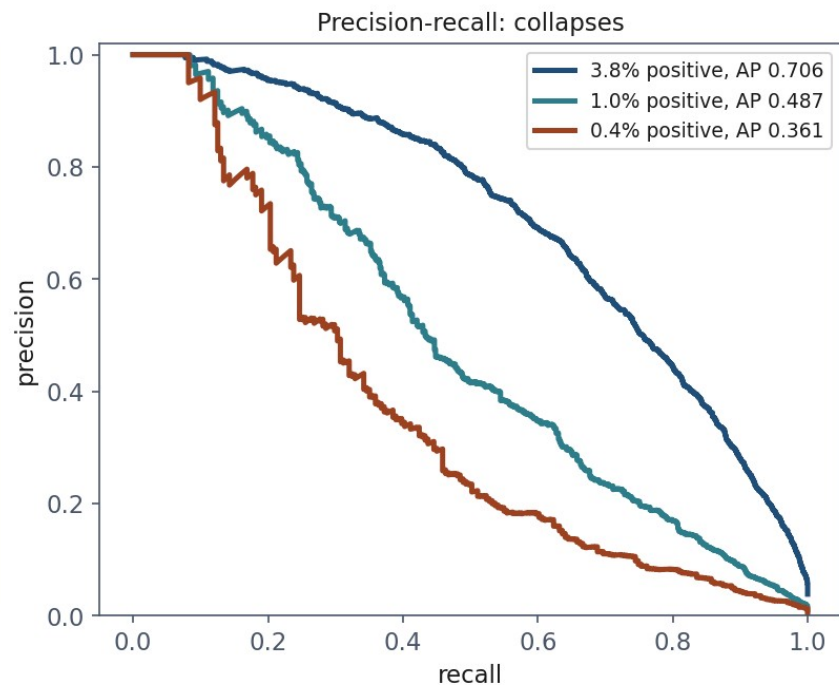
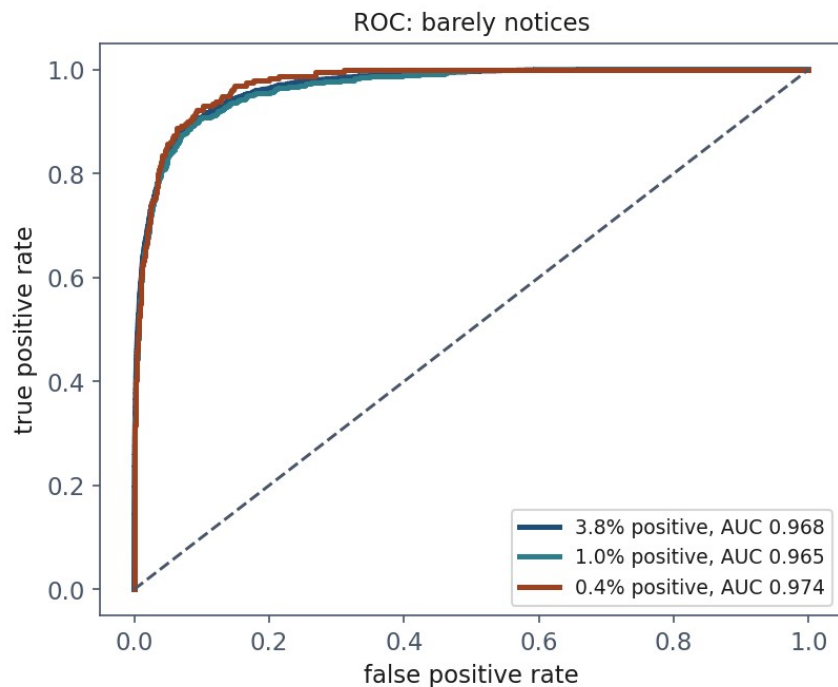
What the area means

- **Area under the curve (AUC) = the probability that a random failing drive scores above a random healthy one**
- Ours: 0.949
- In code: `roc_auc_score(y_test, scores)`

What AUC does not tell you

- Divide every predicted probability by 10
- AUC is **unchanged**; every decision changes

Where ROC misleads



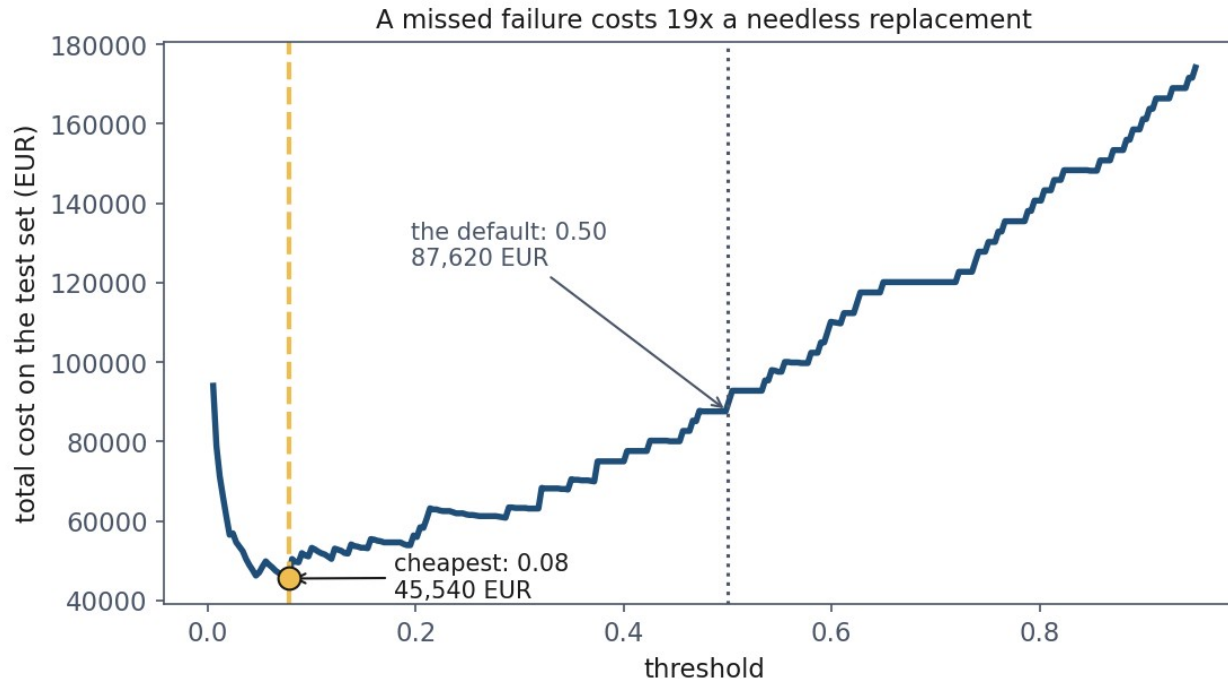
Why the two disagree

- False positive rate (FPR) divides by **all healthy drives**: an enormous, growing denominator
- Precision divides by **the alarms raised**, where the false ones dominate

The rule

- On an imbalanced problem, report the **precision-recall curve alongside AUC**
- Never instead of it, and never AUC alone
- Its area is the **average precision**,
average_precision_score: 0.717 here, against a
floor of 0.038

Choosing the threshold from money



And it has a closed form

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{140}{140 + 2600} = 0.051$$

The caveat you must state

- We chose the threshold **by looking at the test set**
- That is precisely what lesson 1 forbade

Class weights: the same lever, pulled earlier

Model	Recall	Precision	AUC	Cost
Plain, threshold 0.50	0.566	0.768	0.949	87,620
Plain, threshold 0.08	0.855	0.349	0.949	45,540
Balanced class weights, 0.50	0.895	0.249	0.950	49,500

Notebook 3, live

section, and its slides	you call (→ the number to come back with)
the ROC curve by hand, 35	<code>roc_curve</code> , <code>roc_auc_score</code> → 0.9493
what AUC means, 36	200,000 random (failing, healthy) pairs → 0.9488
the rare-positive trap, 38-39	<code>average_precision_score</code> beside <code>roc_auc_score</code> → AP 0.706 to 0.361, AUC unmoved
the threshold from costs, 41-43	<code>confusion_matrix</code> at each threshold → 45,540 EUR at 0.08, against 87,620 at 0.50
class weights, 44	<code>LogisticRegression</code> with balanced class weights → AUC 0.950
three classes, 46-47	<code>classification_report</code> → macro F1 0.707, weighted 0.883

More than two classes



Macro against weighted, one last time

- macro avg F1: **0.707**
- weighted avg F1: **0.883**

What to take away

- **96.20% accuracy, nothing caught.** Accuracy measures the majority class
- Precision is about the **alarms**, recall is about the **failures**
- The **threshold is a decision**, and it has a formula
- **AUC measures ranking** and hides imbalance

Homework

- **Exercise 4**, discussed at the start of **Friday 30 October**
- Build a classifier, choose a threshold from a stated cost ratio, defend it